



EducaTech FIAP Application

09/2026

—

Amanda Abila Melo

RM 373430

Visão geral

O [EducaTech FIAP](#) é uma plataforma de blogging educacional. Professores publicam conteúdos e alunos consultam as postagens.

- **Fase 1:** front-end em OutSystems (telas visuais).
- **Fase 2:** Backend em Node.js + PostgreSQL, com API REST e UI Flows no formato OutSystems.
- **Fase 3 (este projeto):** Interface gráfica em React consumindo a API REST da Fase 2.

Tecnologias do Front-end: React 18, Vite 5, React Router 6, Styled Components 6, Context API, Nginx (produção).

Persona 1 — Ana (Professora): Precisa publicar e atualizar conteúdos rapidamente.

Necessidades: Login simples, área administrativa clara, CRUD de postagens.

Solução no app: Rotas [/login](#), [/admin](#), formulário de criação/edição e confirmação para exclusão.

Persona 2 — Lucas (Aluno): Acessa via dispositivo móvel para estudar.

Necessidades: Lista legível, busca por palavra-chave e leitura sem necessidade de cadastro.

Solução no app: Home pública, busca direcionada (Enter/Clique), página de leitura dedicada e layout responsivo.

Credenciais Padrão (Professor)

- **E-mail:** professor@educatech.com
- **Senha:** [educatech123](#)

Arquitetura do Front-end

Camada	Localização	Responsabilidade
Entrada	src/main.jsx	Inicializa os providers (ThemeProvider , BrowserRouter , AuthProvider).
Rotas	src/App.jsx	Declara a malha de rotas e aplica ProtectedRoute .
Páginas	src/pages/	Visualizações (HomePage , PostPage , LoginPage , AdminPage , etc.).

Componentes	<code>src/components/</code>	Unidades reutilizáveis (<code>Header</code> , <code>PostCard</code> , <code>SearchBar</code> , <code>PostForm</code>).
Estado	<code>src/contexts/</code>	Gerenciamento de sessão com <code>AuthContext</code> .
API	<code>src/services/api.js</code>	Wrapper de requisições <code>fetch</code> e tratamento de erros.
Estilos	<code>src/styles/</code>	Tokens globais, temas e UI base (<code>Styled Components</code>).

Integração com a API REST

Ação na Interface	Método & Rota	Comportamento
Listar posts	<code>GET /posts</code>	Exibe postagens na Home
Filtrar posts	<code>GET /posts/search?q=</code>	Busca por termo preenchido
Detalhar post	<code>GET /posts/:id</code>	Carrega leitura completa
Criar post	<code>POST /posts</code>	Salva novo registro no banco
Editar post	<code>PUT /posts/:id</code>	Atualiza postagem existente
Excluir post	<code>DELETE /posts/:id</code>	Remove registro (retorno <code>204</code>)

Detalhe Técnico: A rota `GET /posts/search` exige o parâmetro `q`. Para evitar erro `400` com busca vazia, a interface só aciona a busca sob confirmação (botão ou Enter).

Autenticação no Cliente

O back-end mantém os endpoints abertos. A autenticação nesta etapa é gerida no client-side:

1. O formulário de login valida credenciais via `AuthContext`.
2. As credenciais são validadas contra variáveis de ambiente (`VITE_AUTH_EMAIL` / `VITE_AUTH_PASSWORD`).
3. O token/estado de sessão é mantido em `localStorage`.
4. `ProtectedRoute` impede o acesso direto às URLs administrativas.

Como usar

Endereço	Uso
http://localhost:3000	API
http://localhost:3000/api-docs	Swagger (testar rotas)
http://localhost:3000/health	Verificar se API e banco estão ok
http://localhost:8080	Frontend

Rodar Localmente (Desenvolvimento)

1. Subir o banco de dados e a API

```
npm install
```

```
docker compose up -d db
```

```
npm start
```

2. Em outro terminal, subir o front-end

```
cd frontend
```

```
npm install
```

```
npm run dev
```

3. Parar:

```
npm run docker:down
```

Dificuldades Encontradas

- **Validação da Busca:** Debug do erro retorno **400** da API em buscas vazias e isolar ações explícitas de envio, em vez de eventos a cada digitação.
- **Normalização de Dados:** Definir o tratamento no front-end para interpretar tanto retornos em array direto quanto objetos paginados.
- **Reutilização de Estados de UI:** Criação de componentes genéricos reduzindo duplicidade de código nas páginas.